APPLY FILTERS TO SQL QUERIES

PORTFOLIO ACTIVITY · SQL FOR SECURITY

ANALYST	ROLE	DATE	REFERENCE
Kelly Guzman	**Founder & SOC Analyst, CTA**	**May 10, 2026**	**Google CC · Course 4**

Project description

In this project, I investigated potential security issues at a large organization by querying the `employees` and `log_in_attempts` tables in the company database. Using SQL filters with `AND`, `OR`, `NOT`, and `LIKE`, I retrieved login activity tied to suspicious events and identified employees whose machines needed security updates. The filtered results gave the security and IT teams the targeted data they needed to act on each issue.

Retrieve after hours failed login attempts

To find failed login attempts that happened after business hours, I filtered the `log_in_attempts` table on two conditions joined with `AND`. Office hours end at 18:00, and the `success` column stores `0` for a failed attempt.

```
SELECT *

FROM log_in_attempts

WHERE login_time > '18:00' AND success = 0;
```

The `AND` operator requires both conditions to be true, so only rows that are both after 18:00 and unsuccessful are returned.

Retrieve login attempts on specific dates

To review all login activity on the day of the suspicious event and the day before, I used `OR` to match either date.

```
SELECT *
```

```
FROM log_in_attempts

WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

The `OR` operator returns a row if either condition is true. Both the date of the event and the day before are included.

Retrieve login attempts outside of Mexico

The `country` column stores Mexico as both `'MEX'` and `'MEXICO'`, so a simple equality check would miss half the records. I used `LIKE` with the wildcard `'MEX%'` to match either spelling, and `NOT` to exclude them.

```
SELECT *

FROM log_in_attempts

WHERE NOT country LIKE 'MEX%';
```

The `%` wildcard matches any characters after `MEX`. `NOT` inverts the filter, returning every login that did not originate in Mexico.

Retrieve employees in Marketing

To find Marketing employees in the East building, I combined two conditions with `AND`. The `office` column contains values like `East-170` and `East-320`, so `LIKE` with the `'East%'` pattern catches all of them.

```
SELECT *

FROM employees

WHERE department = 'Marketing' AND office LIKE 'East%';
```

`AND` ensures only employees who are in Marketing and also in an East building office are returned.

Retrieve employees in Finance or Sales

To pull employees from either of two departments, I used `OR` with two complete conditions. Even though both conditions check the same column, SQL requires the column name to be written on both sides.

```
SELECT *

FROM employees

WHERE department = 'Finance' OR department = 'Sales';
```

`OR` returns a row when either condition is true, so both Finance and Sales employees are included.

Retrieve all employees not in IT

The IT department machines already had the update, so I used `NOT` to exclude them and return everyone else.

```
SELECT *

FROM employees

WHERE NOT department = 'Information Technology';
```

`NOT` reverses the condition. The query returns every employee whose department is anything other than Information Technology.

Summary

This project demonstrated how SQL filters can isolate exactly the records a security team needs from larger tables. I used the `AND` operator to combine conditions for after hours failed logins and Marketing employees in the East building, `OR` to pull records across two dates or two departments, `NOT` with `LIKE` to exclude logins from Mexico across multiple spellings, and `NOT` alone to exclude the IT department. Together these queries support both incident investigation and targeted security updates across the organization.